# CHAPTER 4

# DESIGN DETAILS

## 4.1 Novelty

The proposed system introduces a hybrid approach for coral bleaching analysis by integrating satellite data with real-time observational reports. Unlike existing systems that primarily rely on sea surface temperature (SST) data to detect bleaching events, our approach incorporates ground-level observations to capture the actual condition of coral reefs.

Additionally, the system considers non-thermal environmental parameters such as chlorophyll concentration and water quality indicators including nitrate, nitrite, phosphorus, organic carbon, and total suspended solids. This enables a more comprehensive and accurate assessment of coral bleaching by accounting for both thermal and non-thermal stress factors.

By combining remote sensing data with in-situ observations and multi-parameter analysis, the proposed system improves detection reliability and provides deeper insights into the causes and progression of coral bleaching.

## 4.2 Innovativeness

The CoralWatch system demonstrates innovativeness across its data acquisition pipeline, model design, regularisation strategy, and evaluation methodology.

### 4.2.1 Automated Satellite-to-Prediction Pipeline

The end-to-end pipeline from raw NOAA NetCDF (.nc) satellite files to actionable bleaching predictions is fully automated. The pipeline encompasses on automated data acquisition from NOAA STAR and ERDDAP servers, coordinate-aware GBR spatial cropping (latitude: -24° to -10°, longitude: 142° to 154°); colormap consistent clean PNG generation (without matplotlib

decorations); 4-channel image stacking, BAA label matching and model inference all without manual intervention between stages.

### 4.2.2 Multimodal NLP and Vision Feature Fusion (In Progress)

A parallel NLP pipeline processes coral bleaching research papers and NOAA climate reports to extract text based risk indicators. These text derived features will be fused with CNN visual feature vectors to produce a multimodal prediction. This fusion of structured satellite imagery data with unstructured scientific literature is an innovative approach not previously implemented for coral bleaching early warning.

## 4.3 Interoperability

CoralWatch is designed for maximum interoperability with existing oceanographic data standards and the broader scientific software ecosystem.

The proposed system ensures high compatibility and standardization across multiple components. It directly ingests standard NOAA Coral Reef Watch (CRW) NetCDF4 files (5 km resolution, ct5km format, v3.1) without requiring any pre-conversion, thereby maintaining full compatibility with both current and historical NOAA data products. All spatial processing is performed using the standard WGS84 coordinate reference system, enabling seamless integration with GIS platforms such as ArcGIS and QGIS, as well as satellite imagery and ocean modelling tools. The machine learning models are implemented using PyTorch, a widely adopted deep learning framework, ensuring portability across local systems, cloud environments, and high-performance computing setups. Additionally, all predictions, performance metrics, and outputs are serialized in JSON format to support easy integration with REST APIs, web dashboards, and data pipelines. Training and testing labels are maintained in standard CSV format with date and label fields, allowing compatibility with tools like pandas, R, and Excel. The overall system is built using widely used Python scientific libraries, including xarray, numpy, Pillow, PyTorch, scikit-learn, and matplotlib, ensuring smooth operation across all Python 3.8 and above environments.

## 4.4 Performance

System performance has been rigorously evaluated across prediction accuracy, training efficiency, and inference speed.

### 4.4.1 Prediction Accuracy

The best model (CNN+LSTM with Dropout regularisation) achieves 83.75% test accuracy and 83.47% weighted F1-score on completely unseen 2024–2025 data, predicting BAA alert levels 7 days in advance. This performance is competitive with or superior to published results on similar coral bleaching prediction tasks, particularly given the substantially increased difficulty of 5-class prediction versus binary classification. The per class performance is as follows:

- No Stress (Class 0): Precision 0.82, Recall 0.85, F1-score 0.83
- Bleaching Watch (Class 1): Precision 0.84, Recall 0.71, F1-score 0.77
- Bleaching Warning (Class 2): Precision 0.77, Recall 1.00, F1-score 0.87 , therefore Perfect recall achieved
- Alert Level 1 (Class 3): Precision 1.00, Recall 0.76, F1-score 0.86
- Alert Level 2 (Class 4): Precision 0.84, Recall 0.93, F1-score 0.89

### 4.4.2 Comparative Model Performance

A comprehensive comparison across all evaluated architectures is presented below:

1. CNN+LSTM with Dropout: 83.75% accuracy — Best overall performance
2. CNN+LSTM Baseline: 83.08% accuracy — Competitive but exhibits 7% overfitting gap
3. ConvLSTM Baseline: 81.00% accuracy — High overfitting (20% gap), not recommended
4. ResNet-18 with Early Stopping: 79.73% accuracy — Strongest single-image baseline
5. EfficientNet-B0: 76.71% accuracy — Healthiest training curve, lower accuracy
6. ConvLSTM with Early Stopping: 72.97% accuracy — Weakest performer

### 4.4.3 Training and Inference Efficiency

Training on Google Colab T4 GPU (16 GB VRAM) requires approximately 30 to 60 minutes per model for 30 epochs, with early stopping typically triggering between epochs 9 and 20 depending on the architecture and regularisation strategy. Inference on a single 7-day input sequence requires less than 50 milliseconds on CPU and less than 5 milliseconds on GPU, making the system suitable for near-real-time operational deployment.

## 4.5 Security

Although CoralWatch is a research-oriented environmental monitoring system, appropriate security considerations have been incorporated throughout the design.

The system ensures strong data integrity by sourcing all training data exclusively from official NOAA government servers, including star.nesdis.noaa.gov and coastwatch.pfeg.noaa.gov. This guarantees the authenticity, reliability, and proper provenance of all data used in the model development process.

The proposed system strictly avoids the processing of any personal or sensitive data. It operates solely on publicly available satellite oceanographic datasets, ensuring that no personally identifiable information (PII) is collected, stored, or processed at any stage of the pipeline, thereby maintaining compliance with data privacy standards.

To maintain model security, all trained model weights (.pth files) are stored locally and are not exposed through any public-facing APIs in the current research prototype. This prevents unauthorized access and ensures controlled usage of the trained models.

The data acquisition and preprocessing pipeline incorporates robust input validation mechanisms. It performs checks to detect corrupt, incomplete, or malformed NetCDF files and handles such cases gracefully through error handling and detailed logging, ensuring that invalid data does not impact model predictions.

From a software security perspective, all Python dependencies are sourced from official PyPI repositories. This approach enhances reproducibility while minimizing potential supply chain risks associated with unverified or third-party packages.

For future deployment as a web-based system, standard security practices will be implemented. These include the use of HTTPS for secure communication, authentication mechanisms, rate limiting to prevent abuse, and input sanitization to protect against malicious inputs.

## 4.6 Reliability

The CoralWatch system incorporates multiple mechanisms to ensure reliable and consistent operation across all pipeline stages.

### 4.6.1 Robust Data Pipeline

The data acquisition scripts implement comprehensive error handling with retry logic (up to 5 retries with exponential backoff) for network failures during NOAA satellite data downloads. Corrupted or incomplete NetCDF files are automatically detected and skipped, with a complete log of all failed files produced at the conclusion of each pipeline run. The stacking script validates that all four channels (SST, SSTA, Hotspot, DHW) are available for each date before creating a stacked sample, preventing partial or invalid inputs from entering the training data.

### 4.6.2 Model Checkpoint Reliability

The best-performing model checkpoint (the epoch achieving highest validation accuracy) is saved automatically during training, ensuring that the deployed model consistently represents the optimal trained state regardless of subsequent overfitting in later epochs. Class weights are computed from training label distributions and applied during loss computation, ensuring the model reliably predicts all five BAA alert levels rather than collapsing to majority-class predictions.

### 4.6.3 Evaluation Integrity

Test set evaluation is performed exclusively on 2024–2025 data that was never accessed during training or validation. This strict temporal separation ensures that reported accuracy and F1 metrics reflect genuine out-of-sample generalisation performance and are not inflated by data leakage — a critical reliability requirement for any early warning system intended for operational environmental monitoring deployment.

## 4.7 Maintainability

The system has been designed with long-term maintainability as a primary engineering concern, enabling ongoing development and extension by the project team and future contributors.

The system is designed using a modular pipeline architecture, where the overall processing workflow is divided into clearly defined stages, including data download, NetCDF-to-PNG conversion, image stacking, model training, and evaluation. Each stage is implemented as an independent script, allowing individual components to be updated, replaced, or debugged without impacting the rest of the pipeline, thereby improving maintainability and flexibility.

To simplify configuration management, all key parameters such as years, Great Barrier Reef (GBR) bounding box coordinates, image resolution, forecast horizon, batch size, and learning rate are defined as named constants at the beginning of each script. This centralised approach enables easy system-wide updates by modifying values in a single location without altering the core logic.

The system follows consistent file naming conventions for all generated outputs, such as sst_YYYY_MM_DD.png and stacked_YYYY_MM_DD.npy. This standardisation allows automated file discovery, seamless processing, and easier debugging without requiring additional code changes.

Additionally, the pipeline incorporates comprehensive progress logging mechanisms. Each script provides detailed updates, including progress indicators, success counts, failure reports, and summary statistics, which facilitate efficient monitoring and quick identification of issues without the need for additional debugging tools.

The training workflow is further supported by well-documented Jupyter notebooks. These notebooks are structured with descriptive markdown cells explaining each step, along with clearly organised code cells, enabling team members to easily understand, reproduce, and extend the model training process.

Finally, the system is designed for scalability and future expansion. Incorporating data from additional years, such as 2026 and beyond, requires only updating the predefined YEARS.

configuration constant, after which the entire pipeline automatically processes the new data without further modifications

## 4.8 Portability

CoralWatch has been designed and validated for operation across diverse computing environments, from local development machines to cloud-based training platforms.

- Cross-Platform Python: All pipeline scripts run on macOS, Linux, and Windows operating systems with standard Python 3.8+ and pip-installable dependencies. The project team successfully operated pipelines on macOS Apple Silicon development machines and Linux-based Google Colab environments simultaneously.
- GPU and CPU Flexibility: PyTorch model code automatically detects and utilises available GPU hardware (CUDA) when present, or gracefully falls back to CPU computation. This enables GPU-accelerated training on Google Colab and lightweight CPU-based inference on standard servers.
- Cloud Platform Readiness: The modular script design and standard file formats enable straightforward deployment on major cloud ML platforms including Google Vertex AI, Amazon SageMaker, and Microsoft Azure ML without code modification.
- Container Compatibility: The pipeline can be fully containerised using Docker with a standard PyTorch base image, enabling fully reproducible deployment across any container-capable infrastructure.

- Self-Contained Notebooks: Training notebooks are self-contained with dataset upload, unzipping, training, and result download cells, requiring no pre-installed local software for execution on Google Colab.

## 4.9 Legacy to Modernization

CoralWatch represents a deliberate and systematic modernisation from legacy coral reef monitoring approaches to contemporary deep learning-based early warning systems.

### 4.9.1 From Threshold-Based to Learning-Based Prediction

Traditional coral bleaching monitoring relied on fixed expert-defined DHW thresholds (e.g., DHW > 4 triggers a Watch, DHW > 8 triggers an Alert) to determine bleaching risk. These thresholds cannot adapt to regional variability, inter-variable interactions, or evolving climate patterns. CoralWatch modernises this paradigm by learning complex non-linear multi-variable and multi-temporal patterns directly from data, without relying on manually defined decision boundaries.

### 4.9.2 From Single-Variable to Multi-Channel Representation

Legacy NOAA operational products present SST, SSTA, Hotspot, and DHW as separate independent single-variable maps for expert interpretation. CoralWatch modernises data representation by fusing all four variables into a unified 4-channel image tensor, enabling the model to jointly learn inter-variable relationships and spatial co-occurrence patterns that single-variable systems cannot capture.

### 4.9.3 From Reactive Monitoring to Proactive Forecasting

Traditional reef monitoring products display current or recent historical bleaching conditions, enabling reactive responses after bleaching has commenced. CoralWatch transforms this reactive paradigm into a proactive 7-day predictive early warning system, providing sufficient lead time for

reef management interventions such as targeted cooling, shading, or human activity reduction before bleaching onset.

### 4.9.4 From Manual to Fully Automated Operations

Previous reef monitoring workflows required manual download, format conversion, visualisation, and expert interpretation of satellite data products. CoralWatch automates the entire workflow from NOAA server to BAA alert prediction, enabling operational daily updates with minimal human intervention and consistent, reproducible results.

## 4.10 Reusability

The CoralWatch codebase and architecture have been designed for broad reusability beyond the specific GBR early warning application.

- Generic NetCDF Conversion: The nc_to_png.py conversion script is parameterised by variable name, colormap, and value range through a configuration dictionary. Any NOAA CRW variable can be processed by adding a single dictionary entry, without modifying the core processing logic.
- Reusable PyTorch Dataset Class: The CoralSequenceDataset class accepts any directory of stacked .npy arrays with a corresponding labels CSV, and is fully reusable for any spatiotemporal classification or prediction task without modification.
- Portable Model Architectures: The CNN+LSTM and ConvLSTM architectures are implemented as clean, self-contained nn.Module classes reusable for other multi-channel satellite image time-series prediction tasks beyond coral bleaching.
- Geographic Parameterisation: The pipeline bounding box coordinates are defined as named constants. Applying the system to other coral reef regions (e.g., Florida Reef Tract, Hawaii, Maldives, Coral Triangle) requires only updating four latitude/longitude constants.
- Retrospective Analysis: The pipeline and models can be reused to study any historical bleaching event in the NOAA archive (1985–present), enabling retrospective validation and scientific analysis of past bleaching occurrences.

## 4.11 Application Compatibility

CoralWatch outputs are structured for compatibility with a range of downstream monitoring, decision-support, and research applications.

- Web Dashboard Integration: The prediction output (BAA alert level 0–4, confidence score, per-class probability vector) is structured for direct integration into interactive web-based monitoring dashboards built with React, Vue.js, or standard HTML/CSS/JavaScript.
- GIS Platform Export: Predicted alert levels associated with GBR bounding box coordinates can be exported as GeoJSON or shapefile format for visualisation in QGIS, ArcGIS, and Google Earth Engine, enabling spatial analysis and cartographic presentation.
- NOAA Standards Alignment: Output alert levels follow the established NOAA CRW BAA 0–4 scale, ensuring direct compatibility with existing reef management decision frameworks and operational monitoring protocols.
- NLP Feature Fusion Compatibility: LSTM hidden state vectors are preserved as floating-point feature representations, enabling direct concatenation with NLP-derived risk embeddings in the planned multimodal fusion module.
- REST API Deployment: The inference pipeline can be wrapped in a standard REST API (FastAPI or Flask) exposing a /predict endpoint that accepts a target date parameter and returns a structured JSON response containing the predicted alert level and confidence score.
- Mobile and Field Applications: The low inference latency (< 50 ms on CPU) makes the system suitable for embedding within mobile applications for field use by reef managers and marine biologists.

## 4.12 Resource Utilization

Resource utilisation across storage, computation, memory, and network has been systematically analysed and optimised for both training and deployment phases.

### 4.12.1 Storage Requirements

- Raw NetCDF satellite files: approximately 5–15 MB per file across 2,920 files for 4 variables over 8 years, totalling approximately 60–175 GB for the complete dataset
- Processed PNG images: approximately 50–200 KB per image across 11,680 images, totalling approximately 1–2 GB
- Stacked NumPy arrays: approximately 0.8 MB per file across 2,642 samples (224 × 224 × 4 float32), totalling approximately 2.1 GB
- Trained model weights: approximately 45–80 MB per model in PyTorch .pth format

### 4.12.2 Computational Requirements

- Training: approximately 30–60 minutes per model on Google Colab T4 GPU (16 GB VRAM)
- CNN+LSTM model: approximately 12.5 million trainable parameters
- Training batch size: 16 for CNN+LSTM; 8 for ConvLSTM (constrained by spatiotemporal memory requirements)
- Inference latency: less than 50 ms per 7-day input sequence on CPU; less than 5 ms on GPU

### 4.12.3 Memory Requirements

- Peak GPU memory during training: approximately 8–12 GB, within the 16 GB capacity of Google Colab T4
- Inference RAM requirement: less than 2 GB, suitable for standard cloud server deployment

### 4.12.4 Network Bandwidth

- Data acquisition bandwidth: approximately 5–15 MB per variable per day from NOAA servers
- Operational update bandwidth: approximately 20–60 MB daily for all four satellite variables

- Server rate limiting: 0.5-second delay between successive downloads to comply with NOAA server access policies

### 4.12.5 Applied Optimisations

- Input images resized to 224 × 224 pixels (the standard CNN input resolution) to reduce memory footprint while preserving spatial resolution adequate for GBR-scale bleaching pattern detection
- RGB-to-grayscale channel conversion applied during stacking, reducing per-channel data volume by a factor of three
- NumPy .npy binary format selected over PNG for stacked training arrays to eliminate per-batch image decoding overhead during model training
- PyTorch DataLoader configured with num_workers=2 for parallel asynchronous data loading, preventing GPU starvation between training batches

# CHAPTER 5

# HIGH LEVEL SYSTEM DESIGN /SYSTEM ARCHITECTURE

